

Київський національний університет
імені Тараса Шевченка

Факультет комп'ютерних наук та кібернетики

Звіт

до лабораторної роботи №1
на тему:

**«Розробка клієнта для взаємодії з віддаленою папкою з файлами. Етап 2,
варіант 8-6»**

Студента четвертого курсу
групи МІ-41
Мекшакова Артемія Олексійовича

Київ-2026

Зміст

1	Постановка задачі	1
2	Архітектура реалізації	2
2.1	Структура репозиторію	2
2.2	Відповідність класів UML і коду	2
2.3	Процеси десктоп-клієнта	3
2.4	Уточнення порівняно з UML етапу 1	4
3	Сервер	4
3.1	Модулі	4
3.2	REST API	4
3.3	Нова версія файлу (UC10)	5
3.4	Зберігання	5
3.5	Документація API	5
4	Десктоп-клієнт	5
4.1	Вхід і реєстрація	6
4.2	Простір і таблиця файлів	6
4.3	Сортування за назвою	7
4.4	Фільтр за типом	8
4.5	Показ і приховування стовпців	10
4.6	Перегляд вмісту файлу	10
4.7	Завантаження файлів кнопкою і перетягуванням	11
4.8	Скачування файлів	13
4.9	Видалення файлів	13
4.10	Синхронізація з локальною папкою	13
4.11	Конфлікт версій і його розв'язання	14
4.12	Автоматичне відстеження змін	16
5	Unit-тестування	16
5.1	Обов'язкові тести операцій варіанта	17
5.2	Результати запуску	17
6	Інструкція із запуску	18
7	Висновки	18

1 Постановка задачі

Другий етап лабораторної роботи з курсу «Інформаційні технології» — десктопна версія клієнта системи **типу 2**, тобто програми для роботи з віддаленою папкою з файлами на зразок спрощеного Google Drive. Потрібно створити застосунок із графічним інтерфейсом, написати unit-тести (не менше трьох, по одному на кожну операцію варіанта) і скласти звіт. Разом із клієнтом розроблено сервер Shelf, з яким клієнт обмінюється даними через REST API; веб-клієнт етапу 3 працюватиме з тим самим сервером.

Бонус drag-and-drop виконано в обох напрямках: файли з Finder або Провідника можна кинути

на таблицю, щоб вивантажити їх на сервер, а рядок таблиці — витягнути з вікна, і там, де його відпустили, з'явиться копія файлу.

Реалізація відповідає UML-специфікації етапу 1 і документу дизайну — єдиному джерелу імен: класи з діаграми VOPC мають у коді ті самі назви, ланцюжок повідомлень з діаграми комунікації відтворено викликами функцій, а правила статусів синхронізації записано як чисту функцію `computeStatus`. Уточнення, зроблені під час програмування, зібрано в підрозділі 2.4.

Індивідуальну частину задає варіант **8-6**:

Цифра	Список	Значення
8	ТИПИ файлів, вміст яких показується при кліку	.kt — як текст, .jpg — як зображення
6	ОПЕРАЦІЇ	сортування за назвою (зростання / спадання); фільтр «усі файли» / «лише .cpp» / «лише .png»

2 Архітектура реалізації

2.1 Структура репозиторію

Код зберігається в одному монорепозіторії на `pnpm workspaces`. Спільні пакети підключаються до застосунків як локальні залежності; каталог `apps/web` для веб-версії з'явиться на етапі 3.

`shelf/`

<code>apps/api/</code>	сервер: NestJS 11, Prisma, PostgreSQL, S3
<code>apps/desktop/</code>	десктоп-клієнт: Electron, React, Vite
<code>packages/shared/</code>	@shelf/shared: типи, операції, SyncEngine
<code>packages/ui/</code>	@shelf/ui: React-компоненти клієнтів
<code>docker/</code>	compose: postgres, minio, api

Пакет	Роль	Технології
@shelf/api	REST-сервер: облікові записи, метадані, вміст файлів	NestJS 11, Prisma 6, PostgreSQL 16, S3, JWT, Jest
@shelf/desktop	десктоп-клієнт і синхронізація з папкою	Electron 39, React 19, chokidar, electron-builder, Vitest
@shelf/shared	типи, операції варіанта, перегляд, REST-клієнт, SyncEngine	TypeScript, Vitest; збірка ESM і CommonJS
@shelf/ui	спільні компоненти й хук <code>useFileListController</code>	React 19, Tailwind CSS 4, lucide-react
docker	локальний стек	Docker Compose: PostgreSQL, MinIO, API

2.2 Відповідність класів UML і коду

Таблиця зіставляє класи з діаграм етапу 1 з файлами коду; шлях указано відносно каталогу `src/` пакета. Нових класів, яких немає на діаграмах, два, обидва в `main`-процесі: `SyncService`

тримає план між скануванням і виконанням, керує FolderWatcher і не допускає двох синхронізацій одночасно, а SettingsStore зберігає адресу сервера, прив'язану папку і стан перемикача відстеження.

Клас з діаграм етапу 1	Реалізація	Пакет	Файл
SortHeaderControl	заголовок стовпця «Назва»	@shelf/ui	SortHeaderControl.tsx
TypeFilterControl	три кнопки фільтра	@shelf/ui	TypeFilterControl.tsx
FileTableView	таблиця файлів	@shelf/ui	FileTableView.tsx
FileListController	хук useFileListController	@shelf/ui	useFileListController.ts
FileApiClient	REST-клієнт	@shelf/shared	fileApiClient.ts
FileListOperations	sortByName, filterByType, getVisibleFiles	@shelf/shared	fileListOperations.ts
FilePreview, TextPreview, ImagePreview	класи, previewKindOf, createPreview	@shelf/shared	preview.ts
PreviewDialog	модальне вікно перегляду	@shelf/ui	PreviewDialog.tsx
SyncEngine	рушій синхронізації	@shelf/shared	sync/syncEngine.ts
SyncItem	тип, computeStatus, decideDirection	@shelf/shared	sync/syncPlanner.ts
SyncSnapshot, SnapshotEntry	знімок, інтерфейс SnapshotStore	@shelf/shared	sync/snapshot.ts
SnapshotStore	JsonSnapshotStore	@shelf/desktop	main/jsonSnapshotStore.ts
LocalFolder	інтерфейс, MemoryLocalFolder	@shelf/shared	sync/localFolder.ts
LocalFolder	NodeLocalFolder на Node fs	@shelf/desktop	main/nodeLocalFolder.ts
SyncPanel	блок синхронізації	@shelf/ui	SyncPanel.tsx
SyncReport	тип звіту	@shelf/shared	sync/syncEngine.ts
SyncReportView	показ звіту	@shelf/ui	SyncReportView.tsx
ConflictDialog	діалог вибору версії	@shelf/ui	ConflictDialog.tsx
FolderWatcher	спостерігач на chokidar	@shelf/desktop	main/folderWatcher.ts
DragOutHandler	FileTransfers	@shelf/desktop	main/fileTransfers.ts

2.3 Процеси десктоп-клієнта

Поділ Electron на процеси збігається з межами відповідальності. **Main-процес** (src/main) має доступ до файлової системи, тож у ньому працюють SettingsStore, NodeLocalFolder, JsonSnapshotStore, FolderWatcher, SyncService і FileTransfers, а registerIpc оголошує обробники запитів. **Renderer-процес** (src/renderer) — сторінка React з екранами LoginScreen і WorkspaceScreen, ізольована від Node (contextIsolation, sandbox); список, вивантаження, перегляд і видалення вона виконує сама через FileApiClient. **Preload-скрипт** (src/preload) дає їй типізований міст window.shelf (ShelfBridge): кожен метод — це ipcRenderer.invoke, а прогрес синхронізації надходить подіями IPC.

Сортування, фільтр і план синхронізації обчислюються на клієнті. Сервер віддає простий список, а клієнт упорядковує й фільтрує його в пам'яті, тому перемикач справцює миттєво. Ці чисті функції в @shelf/shared перевіряються unit-тестами без сервера і без змін підійдуть веб-клієнту. План синхронізації сервер скласти й не може: для нього потрібні локальна папка і знімок стану.

2.4 Уточнення порівняно з UML етапу 1

- `decideDirection(item)` — функція планувальника в `syncPlanner.ts`, а не метод `SyncItem`: елемент плану лишився простим записом даних, що проходить через IPC.
- Спостереження за папкою винесено з `SyncEngine` у десктопні `SyncService` і `FolderWatcher`, тож рушій не залежить від платформи й підійде для веба.
- `SnapshotEntry.remoteId` прив'язує запис знімка до `id` серверного файлу. Запис про інший файл під тим самим ім'ям (інший обліковий запис, скинутий сервер) не враховується, і файли порівнюються за контрольною сумою.
- Віддалена зміна визначається також за `checksum`, тож нову версію, вивантажену в межах допуску часу 2 с, теж помічено.
- План, складений викликом `scan()`, перед перенесенням перевіряється ще раз; файл, змінений після перевірки, пропускається з повідомленням у звіті.
- Імена, що відрізняються лише регістром або формою `Unicode` (`Main.kt` і `main.kt`), на `macOS` і `Windows` позначають один файл, тому пропускаються з повідомленням у звіті.
- `resolve(name, keep)` приймає назву файлу, а не `SyncItem`: назва є ключем плану і легко передається між процесами.
- До REST API додано `GET /api/health` для перевірки стану контейнера сервера.

3 Сервер

3.1 Модулі

Сервер — застосунок `NestJS 11` з префіксом `/api`. `AuthModule` (`AuthController`, `AuthService`, `JwtStrategy`, `JwtAuthGuard`) реєструє користувачів і видає JWT на 24 години; `UsersModule` створює користувачів разом з просторами; `WorkspaceModule` визначає простір поточного користувача; `FilesModule` відповідає за вивантаження, нові версії, скачування й видалення; `StorageModule` працює з бакетом через `S3 API`; `PrismaModule` дає доступ до бази. Є також `ConfigModule` для перевірки змінних середовища і `HealthController`. Тіла запитів перевіряє `class-validator`, а повідомлення про помилки сервер повертає українською.

3.2 REST API

Усі шляхи мають префікс `/api`; токен `Authorization: Bearer` потрібен скрізь, крім реєстрації, входу і перевірки стану.

Метод	Шлях	Тіло / параметри	Відповідь
POST	<code>/auth/register</code>	<code>RegisterDto</code>	<code>AuthResponseDto</code>
POST	<code>/auth/login</code>	<code>LoginDto</code>	<code>AuthResponseDto</code>
GET	<code>/auth/me</code>	—	<code>UserDto</code>
GET	<code>/workspace</code>	—	<code>WorkspaceDto</code> (<code>id</code> , <code>owner</code> , <code>fileCount</code>)
GET	<code>/workspace/files</code>	—	<code>FileEntryDto[]</code>
POST	<code>/workspace/files</code>	<code>multipart</code> , поле <code>file</code>	<code>FileEntryDto</code> ; 201 — новий файл, 200 — нова версія

Метод	Шлях	Тіло / параметри	Відповідь
GET	/workspace/files/:id	—	FileEntryDto
GET	/workspace/files/:id/content	—	вміст файлу з типом і назвою в заголовках
DELETE	/workspace/files/:id	—	204
GET	/health	—	{status: "ok"}

Коди помилок: 400 — некоректне тіло або назва файлу, 401 — немає чи прострочений токен, 404 — файлу немає в просторі користувача (так само для чужого файлу і некоректного id), 409 — адресу пошти вже зареєстровано, 413 — файл понад 50 МБ.

3.3 Нова версія файлу (UC10)

Вивантаження файлу з назвою, яка вже є в просторі, створює нову версію. `FileService.upsert` шукає запис за унікальною парою «простір, назва». Лишаються той самий `FileEntry` з його id, датою створення, автором і ключем у сховищі, а змінюються байти, size, checksum (SHA-256, рахує сервер), `modifiedAt` (поточний час) і `editedBy` — поточний користувач, тобто власник простору; відповідь має код 200 замість 201. Назва зберігається точно такою, якою її надіслав клієнт, без обрізання пробілів і нормалізації.

3.4 Зберігання

Метадані лежать у PostgreSQL 16 (моделі Prisma User, Workspace, FileEntry), а байти — в S3-сумісному сховищі: локально це MinIO в Docker Compose, після публікації — хмарне сховище з тим самим S3 API. Ключ об'єкта має вигляд `workspaces/{workspaceId}/{id}` у бакеті `shelf`, який `StorageService` за потреби створює під час запуску. Вміст віддається потоком, `Content-Type` визначається за розширенням, а `Content-Disposition` зберігає кириличну назву файлу.

3.5 Документація API

За адресою `/api/docs` сервер показує Swagger UI з описом API у форматі OpenAPI (змінна `SWAGGER_ENABLED=false` вимикає цю сторінку, наприклад на публічному сервері). На рис. 1 видно всі маршрути, згруповані за контролерами `health`, `auth`, `workspace` і `files`; замок біля маршруту означає, що потрібен токен, а кнопка «Authorize» дає змогу вставити JWT і викликати захищені маршрути з браузера. Унизу наведено схеми `RegisterBody` і `LoginBody`, за якими перевіряються реєстрація та вхід. Сторінка дає змогу перевірити вимоги R1 і R7 на рівні сервера без клієнта.

4 Десктоп-клієнт

Ліворуч у вікні — темна бічна панель, праворуч — панель інструментів і таблиця; перегляд і конфлікти відкриваються в модальних вікнах. Знімки зроблено на macOS під обліковим записом «Артем».

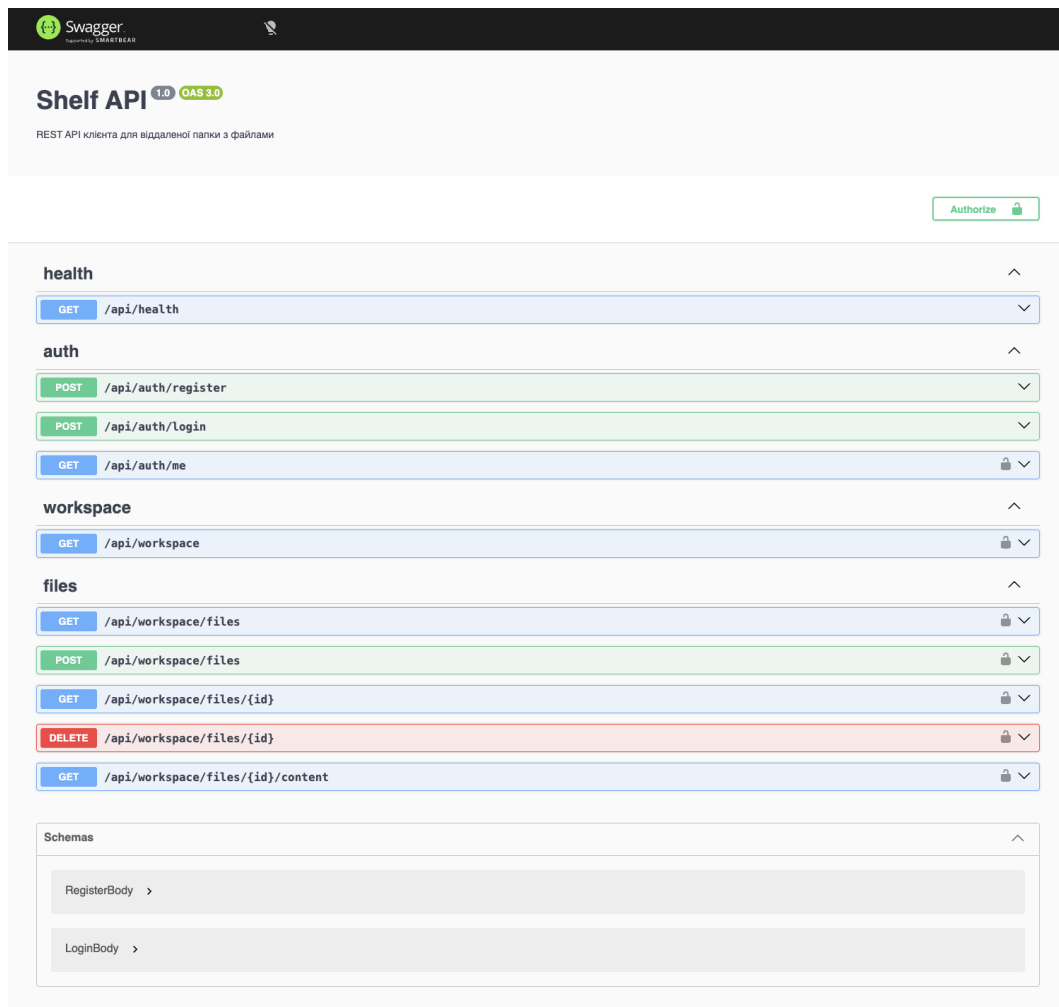


Рисунок 1. Swagger UI сервера зі списком маршрутів REST API

4.1 Вхід і реєстрація

Екран входу (рис. 2) реалізує вимогу R1 і прецедент UC2. Над полями пошти й пароля стоїть поле «Адреса сервера» зі значенням `http://localhost:4000`; після успішного входу адреса зберігається в налаштуваннях, тож клієнт можна під'єднати й до іншого сервера, зокрема до публічного на етапі 3. Посилання під кнопкою відкриває реєстрацію (рис. 3, UC1) з додатковим полем «Ім'я»: сервер створює користувача з порожнім простором і відразу повертає токен. Під час наступного запуску клієнт перевіряє збережений токен запитом `GET /api/auth/me` і лише після відмови 401 показує вхід знову. Кнопка «Вийти» завершує сесію (UC15).

4.2 Простір і таблиця файлів

Після входу відкривається простір користувача (рис. 4) — вимога R2, прецеденти UC3 і UC4. Таблиця містить десять демонстраційних файлів у порядку зростання назв. Для кожного показано назву з піктограмою типу, розширення, розмір, дату й час створення та зміни, хто завантажив і хто редагував, а також кнопки скачування й видалення. Лічильник «10 з 10» порівнює видимі рядки з усіма файлами, «Оновити» заново запитує список. Кожен користувач має власний простір, тому в стовпцях «Хто завантажив» і «Хто редагував» стоїть власник —

Рисунок 2. Екран входу з полем адреси сервера

Рисунок 3. Форма реєстрації нового користувача

«Артем»; різні імена з'явилися б лише за спільних папок, які поза межами проєкту (§1.3 і §9 специфікації).

Ізоляцію просторів демонструють три демо-користувачі — Артем, Ірина й Максим — з різними наборами файлів: запит чужого файлу за його id повертає 404, так само як запит файлу, якого не існує.

4.3 Сортування за назвою

Сортування — перша операція варіанта (вимога R4, прецедент UC5). Клік по заголовку «Назва» (SortHeaderControl) перемикає напрямок, а піктограма показує його: «А над Z» для зростання, «Z над А» для спадання. На рис. 5 список упорядковано за спаданням: першим іде Звіт.txt, останнім — backup.zip. Хук useFileListController зберігає напрямок і буде видимий список за інваріантом `filterByType(sortByName(files, direction), filter)`. Порівняння виконує Intl.Collator з мовою системи: регістр не враховується, числа в назвах ідуть у природному порядку (file2 перед file10). Від мови системи залежить і

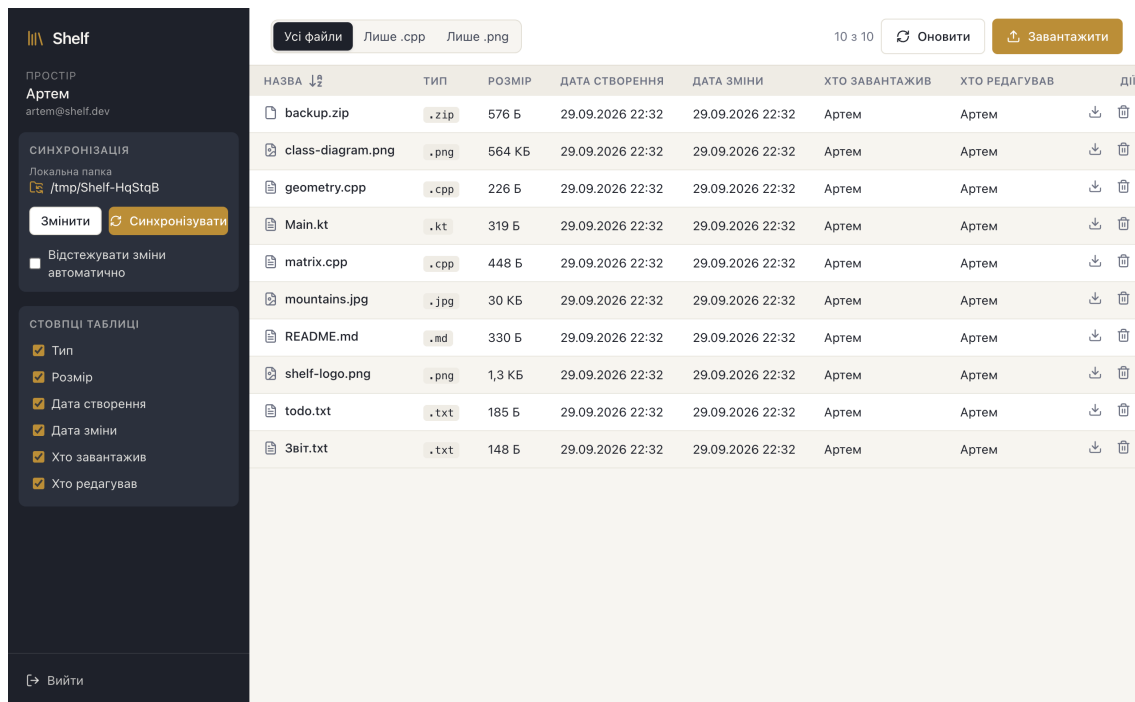


Рисунок 4. Простір користувача: таблиця файлів з усіма атрибутами

місце кириличних назв: на англomовній, як на знімках, вони стоять після латинських, на україномовній — перед ними. Запитів до сервера при цьому немає.

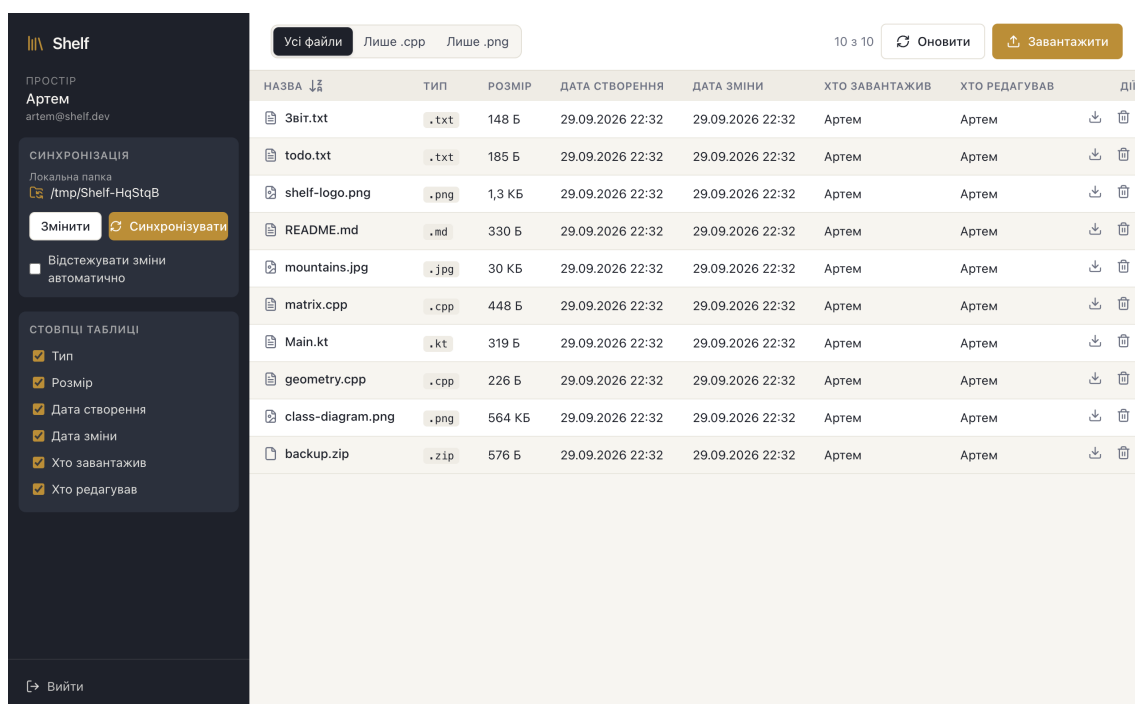


Рисунок 5. Файли, відсортовані за назвою за спаданням

4.4 Фільтр за типом

Фільтр — друга операція варіанта (вимога R5, прецедент UC6). `TypeFilterControl` — група кнопок «Усі файли», «Лише .cpp» і «Лише .png»; вибрана кнопка темна. На рис. 6 увімкнено

«Лише .cpp»: лишилися `geometry.cpp` і `matrix.cpp`, лічильник показує «2 з 10». Порівнюється саме розширення без урахування регістру, тож `Zeta.CPP` проходить фільтр, а `readme-cpp.txt` — ні. Кнопка «Усі файли» повертає повний список; як і сортування, фільтрація відбувається на клієнті без запиту до сервера.

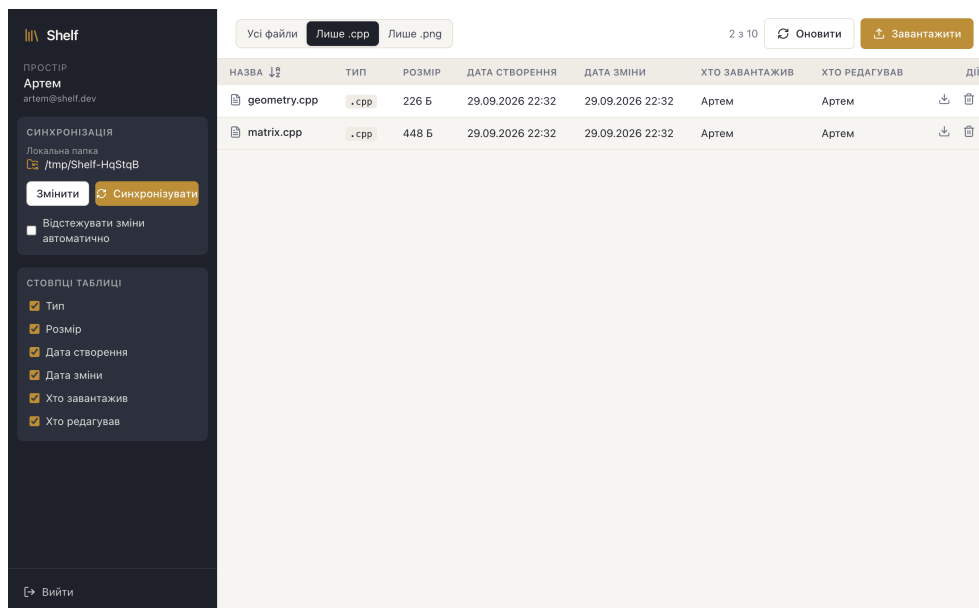


Рисунок 6. Фільтр «Лише .cpp»

На рис. 7 вибрано «Лише .png», і видно `class-diagram.png` та `shelf-logo.png`. Фільтр застосовується поверх відсортованого списку, тому його зміна не скидає обраного порядку: після сортування за спаданням `.cpp`-файли теж ідуть від `matrix.cpp` до `geometry.cpp`, що перевіряє наскрізний тест інтерфейсу.

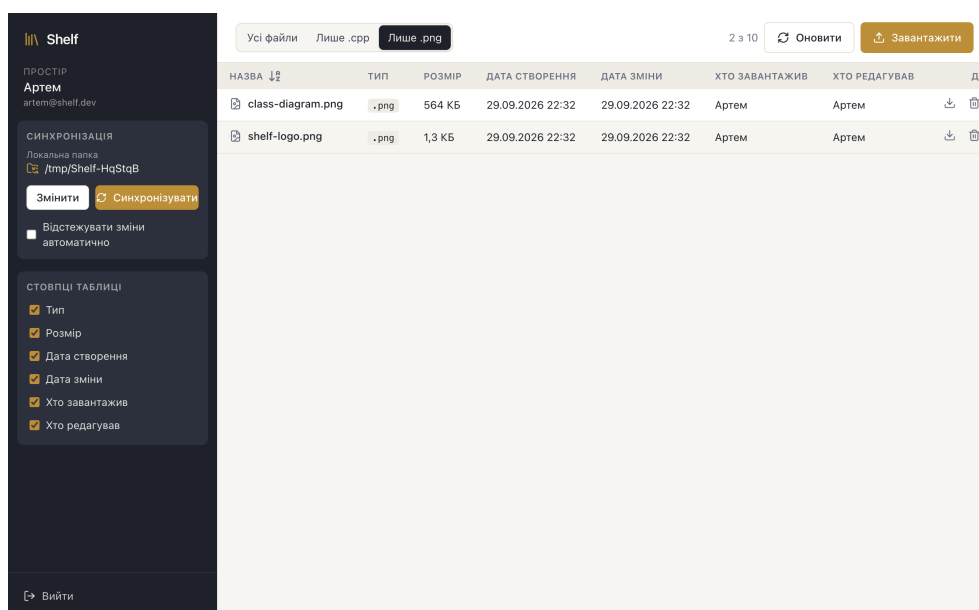
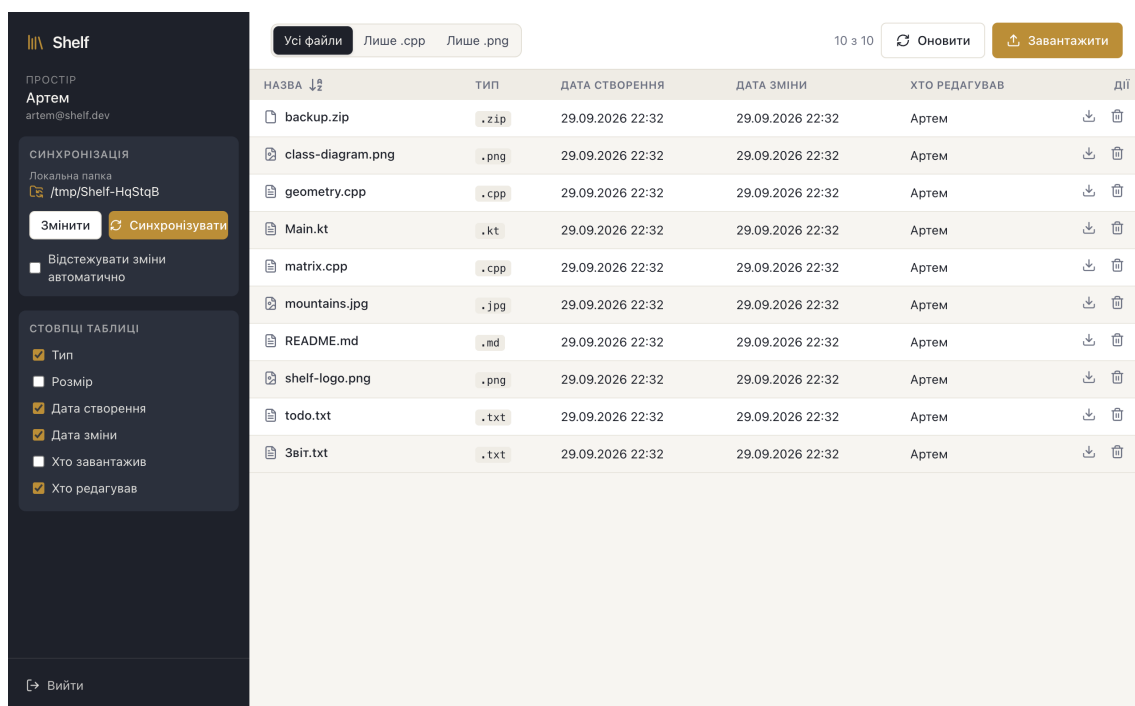


Рисунок 7. Фільтр «Лише .png»

4.5 Показ і приховування стовпців

Блок «Стовпці таблиці» в бічній панелі (ColumnPicker) реалізує вимогу R3 і прецедент UC7: кожен стовпець, крім назви, має свій прапорець. На рис. 8 вимкнено «Розмір» і «Хто завантажив», і таблиця одразу втратила ці стовпці, а решта атрибутів лишилася. Стовпця «Назва» в переліку немає: без нього рядок втратив би сенс, тому `toggleColumn` із `@shelf/shared` ігнорує спробу його сховати, що перевіряє unit-тест. Видимість стовпців зберігається в стані `useFileListController` і впливає лише на відображення, дані заново не запитуються.



НАЗВА ↓	ТИП	ДАТА СТОВРЕННЯ	ДАТА ЗМІНИ	ХТО РЕДАГУВАВ	ДІЇ
backup.zip	.zip	29.09.2026 22:32	29.09.2026 22:32	Артем	⬇️ 🗑️
class-diagram.png	.png	29.09.2026 22:32	29.09.2026 22:32	Артем	⬇️ 🗑️
geometry.cpp	.cpp	29.09.2026 22:32	29.09.2026 22:32	Артем	⬇️ 🗑️
Main.kt	.kt	29.09.2026 22:32	29.09.2026 22:32	Артем	⬇️ 🗑️
matrix.cpp	.cpp	29.09.2026 22:32	29.09.2026 22:32	Артем	⬇️ 🗑️
mountains.jpg	.jpg	29.09.2026 22:32	29.09.2026 22:32	Артем	⬇️ 🗑️
README.md	.md	29.09.2026 22:32	29.09.2026 22:32	Артем	⬇️ 🗑️
shelf-logo.png	.png	29.09.2026 22:32	29.09.2026 22:32	Артем	⬇️ 🗑️
todo.txt	.txt	29.09.2026 22:32	29.09.2026 22:32	Артем	⬇️ 🗑️
3bit.txt	.txt	29.09.2026 22:32	29.09.2026 22:32	Артем	⬇️ 🗑️

Рисунок 8. Приховано стовпці «Розмір» і «Хто завантажив»

4.6 Перегляд вмісту файлу

Клік по рядку відкриває модальне вікно `PreviewDialog` — вимога R6, прецедент UC8, тип варіанта. Спосіб перегляду визначає чиста функція `previewKindOf` за розширенням; `createPreview` створює `TextPreview` або `ImagePreview`, клієнт скачує вміст через `FileApiClient.download`, а `render` готує результат. На рис. 9 файл `Main.kt` показано як текст моноширинним шрифтом, над ним — розмір, дати, автор і редактор. З великого текстового файлу показується перший мегабайт. Вікно не виходить за межі екрана: заголовок і кнопки закріплені, вміст прокручується.

Файл `mountains.jpg` на рис. 10 відкривається як зображення, масштабоване так, щоб поміститися у вікно разом з атрибутами. Варіант вимагає лише `.kt` і `.jpg`, але перегляд зроблено узагальнено, бо типи перегляду не збігаються з типами фільтра: будь-який текстовий файл (`.kt`, `.cpp`, `.txt`, `.md`, `.json` тощо) показується як текст, будь-яке растрове зображення (`.jpg`, `.jpeg`, `.png`, `.gif`, `.webp`, `.bmp`) — як картинка.

Для решти типів, як-от архіву `backup.zip` на рис. 11, вікно показує тільки атрибути і повідомлення «Перегляд недоступний для цього типу файлу»; вміст такого файлу клієнт

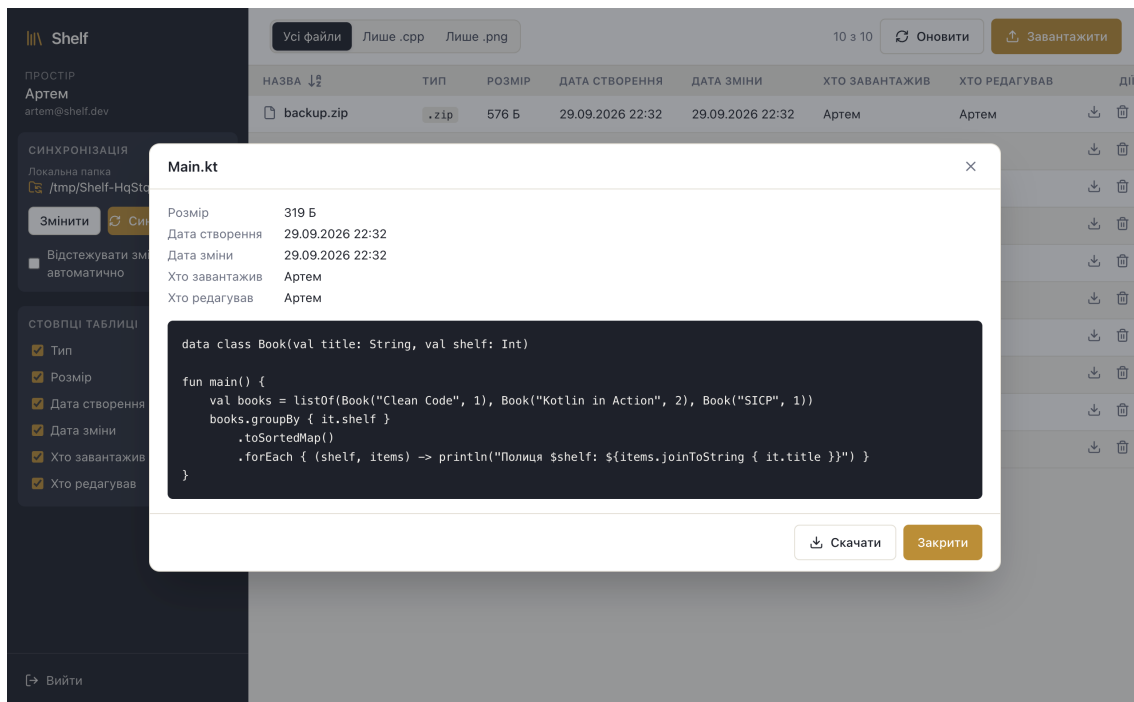


Рисунок 9. Перегляд файлу .kt як тексту

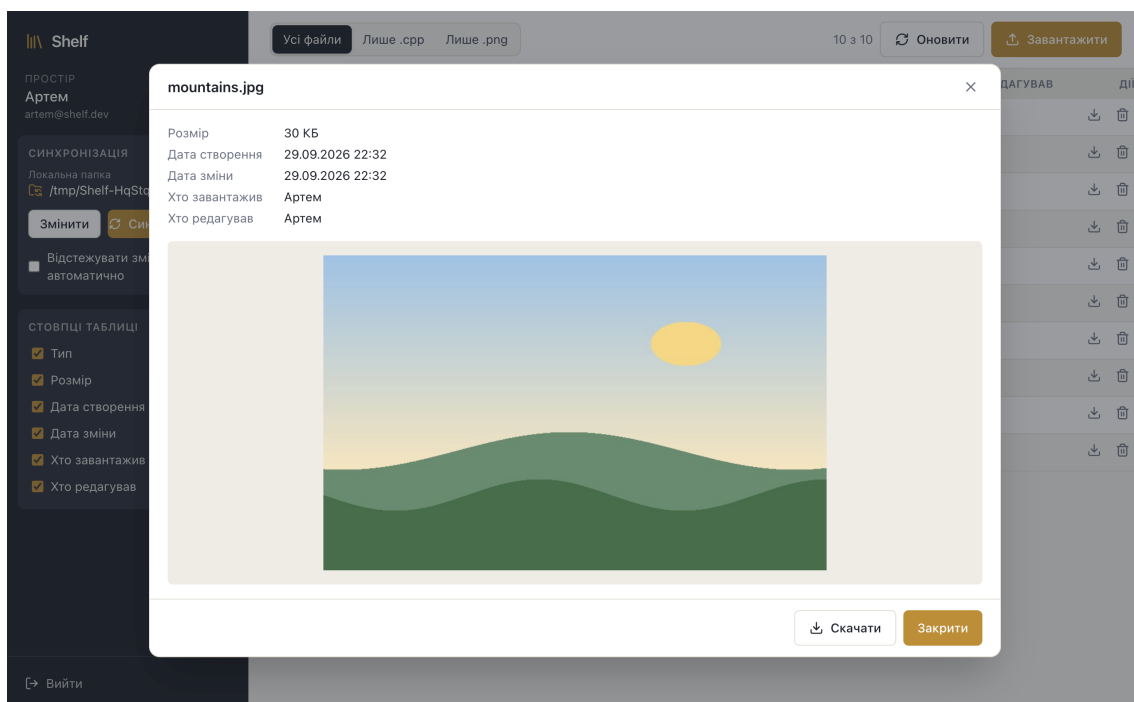


Рисунок 10. Перегляд файлу .jpg як зображення

навіть не скачує. Кнопка «Скачати» зберігає файл на диск, а закрити вікно можна кнопкою «Закрити», клавішею Escape або кліком поза ним.

4.7 Завантаження файлів кнопкою і перетягуванням

Вивантаження на сервер покриває вимоги R7 і R9, прецеденти UC9a і UC9b. Кнопка «Завантажити» відкриває системний діалог з вибором кількох файлів. Можна й перетягнути файли з Finder чи Провідника на таблицю: UploadDropzone накриває її шаром з пунктирною

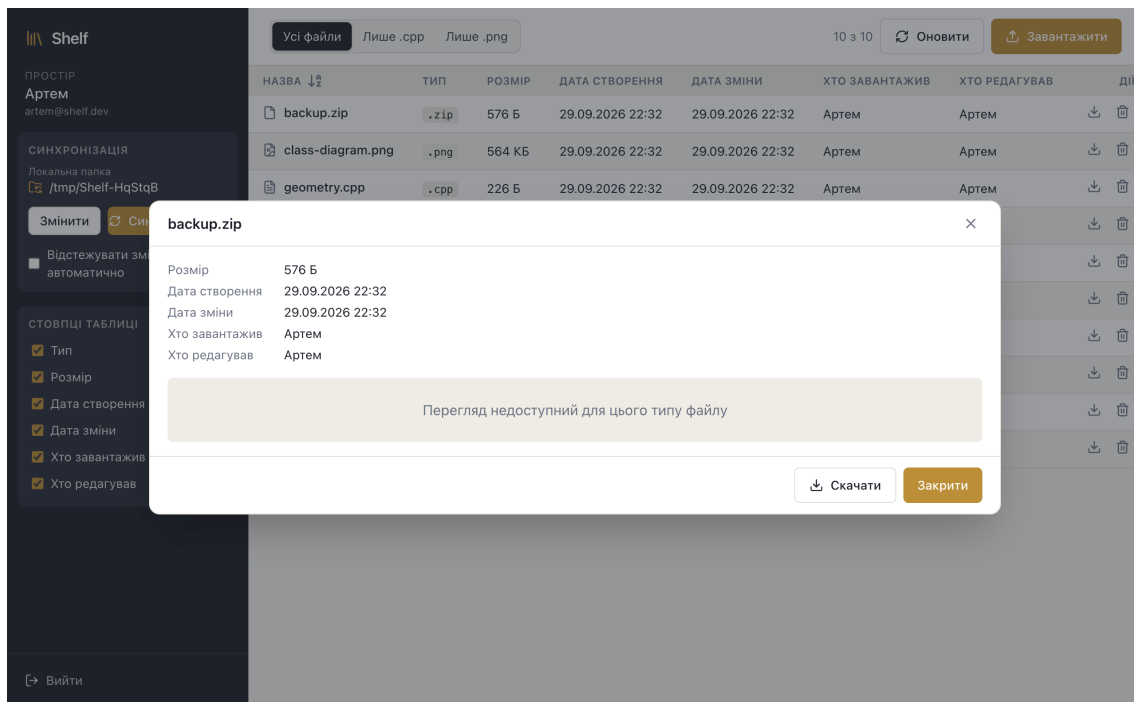


Рисунок 11. Файл без перегляду: лише атрибути

рамкою і написом «Відпустіть файли, щоб завантажити їх у простір» (рис. 12). Шар реагує тільки на перетягування з файлами операційної системи (у `dataTransfer.types` є `Files`), а поки користувач тягне рядок таблиці з вікна, зона вимкнена, тож шар тоді не з'являється.

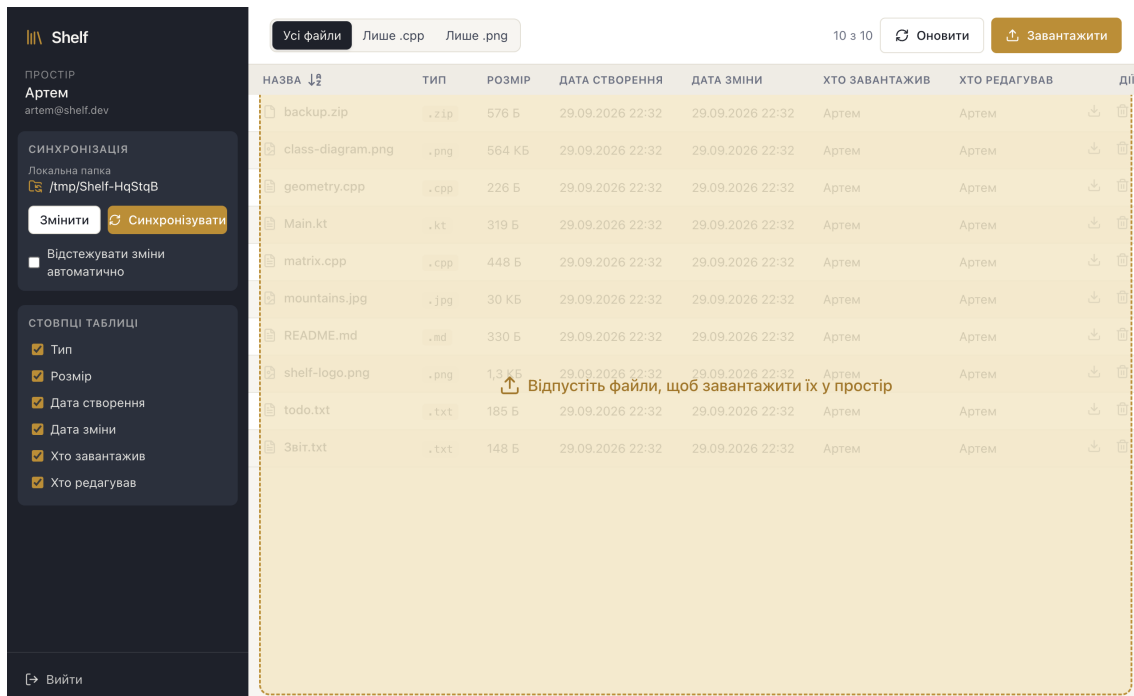


Рисунок 12. Файл перетягують у вікно

Після того як файл відпущено (рис. 13), `Greeting.kt` з'явився в таблиці, лічильник показує «11 з 11», а повідомлення — «Завантажено файлів: 1». Обмеження 50 МБ перевіряється двічі: клієнт функцією `splitBySize` відкладає завеликі файли ще до запиту й називає їх у повідомленні

«Більші за 50 МБ і пропущені: ...», а решту вивантажує; сервер додатково відповідає на такий файл кодом 413. Файл із назвою, яка вже є в просторі, стає його новою версією (UC10).

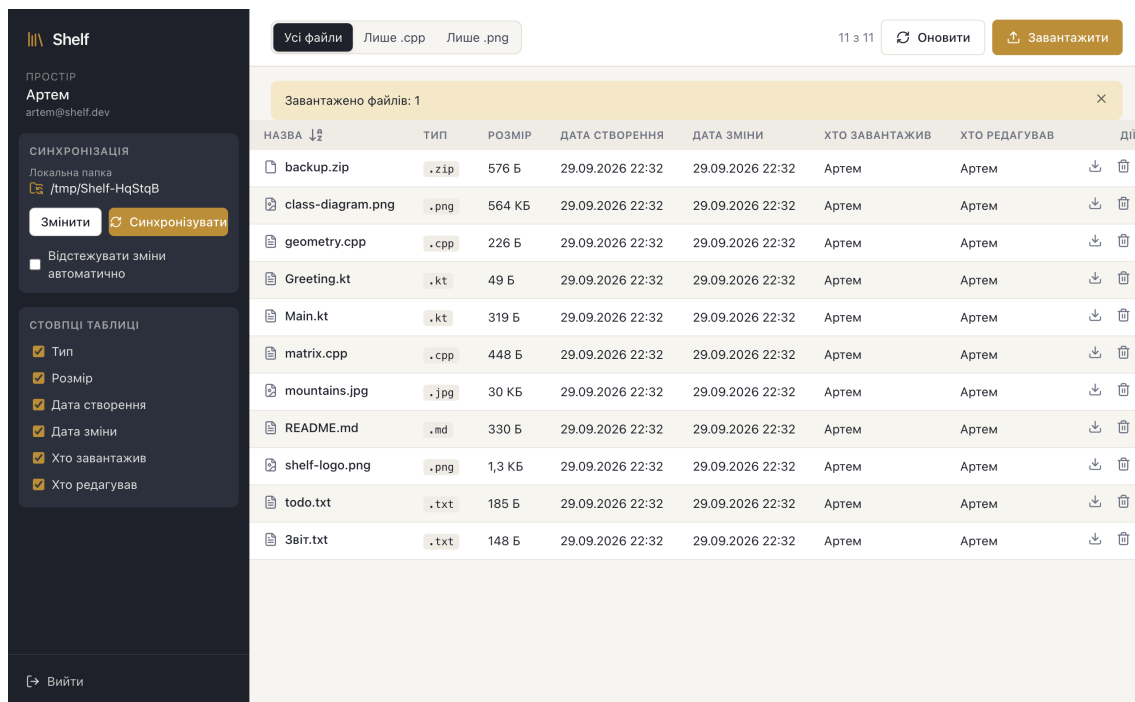


Рисунок 13. Результат перетягування: файл вивантажено

4.8 Скачування файлів

Скачування (UC11) має два способи. Кнопка зі стрілкою в рядку або «Скачати» у вікні перегляду відкриває системний діалог збереження файлу. Другий спосіб — витягнути рядок з вікна у Finder або Провідник (UC11a, бонус R9). Перетягування в браузері не створює файл на диску, тому використано механізм Electron `webContents.startDrag`. Щойно користувач натискає кнопку миші на рядку, `main`-процес на прохання сторінки (`prepareDrag`) скачує файл у тимчасовий каталог `shelf-drag/<id>/<назва>`. Коли починається перетягування, сторінка скасовує стандартну дію браузера, вимикає зону вивантаження і викликає `startDrag`; `main`-процес передає системі шлях до файлу й піктограму, і файл копіюється туди, де користувач відпустив кнопку.

4.9 Видалення файлів

Кнопка з кошиком видаляє файл (UC12): після підтвердження клієнт надсилає `DELETE /api/workspace/files/:id`, сервер видаляє запис і об'єкт зі сховища, а таблиця перечитує список. Синхронізація видалень не поширює: файл, видалений на сервері, але залишений у прив'язаній папці, наступна синхронізація вивантажить знову — це задокументоване обмеження.

4.10 Синхронізація з локальною папкою

Блок «Синхронізація» в бічній панелі реалізує вимогу R8 і прецеденти UC13 та UC14. У ньому видно шлях прив'язаної папки, кнопку «Змінити» для вибору іншої, кнопку «Синхронізувати» і

прапорець автоматичного відстеження; шлях зберігається в налаштуваннях. «Синхронізувати» запускає в main-процесі `scan()`, що порівнює папку, список на сервері і знімок стану, а потім виконання плану з індикатором прогресу. На рис. 14 показано першу синхронізацію папки, де лежав лише `lab-notes.txt`: «Завантажено на сервер: 1», «Скачано з сервера: 11», «Без змін: 0», «Конфліктів: 0», а в таблиці з’явився новий файл. Скачаним файлам на диску виставляється час зміни серверної версії, тож наступна перевірка вважає їх незміненими.

The screenshot shows the Shelf application interface. On the left, a sidebar contains a 'СИНХРОНІЗАЦІЯ' (Synchronization) section with a status summary: 'Завантажено на сервер: 1' (Uploaded to server: 1), 'Скачано з сервера: 11' (Downloaded from server: 11), 'Без змін: 0' (No changes: 0), and 'Конфліктів: 0' (Conflicts: 0). Below this is a table of columns to be displayed in the file list.

НАЗВА	ТИП	РОЗМІР	ДАТА СТВОРЕННЯ	ДАТА ЗМІНИ	ХТО ЗАВАНТАЖИВ	ХТО РЕДАГУВАВ	ДІЇ
backup.zip	.zip	576 Б	29.09.2026 22:32	29.09.2026 22:32	Артем	Артем	Download, Delete
class-diagram.png	.png	564 КБ	29.09.2026 22:32	29.09.2026 22:32	Артем	Артем	Download, Delete
geometry.cpp	.cpp	226 Б	29.09.2026 22:32	29.09.2026 22:32	Артем	Артем	Download, Delete
Greeting.kt	.kt	49 Б	29.09.2026 22:32	29.09.2026 22:32	Артем	Артем	Download, Delete
lab-notes.txt	.txt	95 Б	29.09.2026 22:32	29.09.2026 22:32	Артем	Артем	Download, Delete
Main.kt	.kt	319 Б	29.09.2026 22:32	29.09.2026 22:32	Артем	Артем	Download, Delete
matrix.cpp	.cpp	448 Б	29.09.2026 22:32	29.09.2026 22:32	Артем	Артем	Download, Delete
mountains.jpg	.jpg	30 КБ	29.09.2026 22:32	29.09.2026 22:32	Артем	Артем	Download, Delete
README.md	.md	330 Б	29.09.2026 22:32	29.09.2026 22:32	Артем	Артем	Download, Delete
shelf-logo.png	.png	1,3 КБ	29.09.2026 22:32	29.09.2026 22:32	Артем	Артем	Download, Delete
todo.txt	.txt	185 Б	29.09.2026 22:32	29.09.2026 22:32	Артем	Артем	Download, Delete
Zeit.txt	.txt	148 Б	29.09.2026 22:32	29.09.2026 22:32	Артем	Артем	Download, Delete

Рисунок 14. Результат першої синхронізації

4.11 Конфлікт версій і його розв’язання

Після кожного перенесення клієнт записує в знімок стану `SnapshotEntry`: час і розмір локального файлу, час, контрольну суму і `remoteId` серверної версії. Знімок зберігається у `userData/snapshots/<sha1 шляху папки>.json`, окремо для кожної папки. Файл лише з одного боку має статус `LOCAL_ONLY` або `REMOTE_ONLY`; файл з обох боків порівнюється зі знімком: зміна лише локально дає `LOCAL_NEWER`, лише на сервері — `REMOTE_NEWER`, з обох боків — `CONFLICT`, інакше `IN_SYNC`. «Змінилося локально» — час поза допуском 2 с або інший розмір; «змінилося віддалено» — `modifiedAt` поза допуском 2 с або інший checksum; запис знімка іншого серверного файлу (`remoteId`) не враховується (§5.2). Без знімка вирішує контрольна сума.

На рис. 15 файл `todo.txt` змінено і в папці (22:27, 80 Б), і на сервері (22:32, 66 Б), тому відкривається `ConflictDialog` (UC14b). Для кожного файлу можна залишити локальну версію або версію з сервера; новішу вибрано заздалегідь. «Застосувати» передає вибір у `run(resolutions)`, а «Скасувати синхронізацію» нічого не переносить, відкидає план і повертає роботу автоматичному відстеженню. Файл, змінений, поки було відкрито діалог, під час повторної перевірки плану пропускається.

Рис. 16 показує результат: версію з сервера скачано, звіт містить «Скачано з сервера: 1», «Без

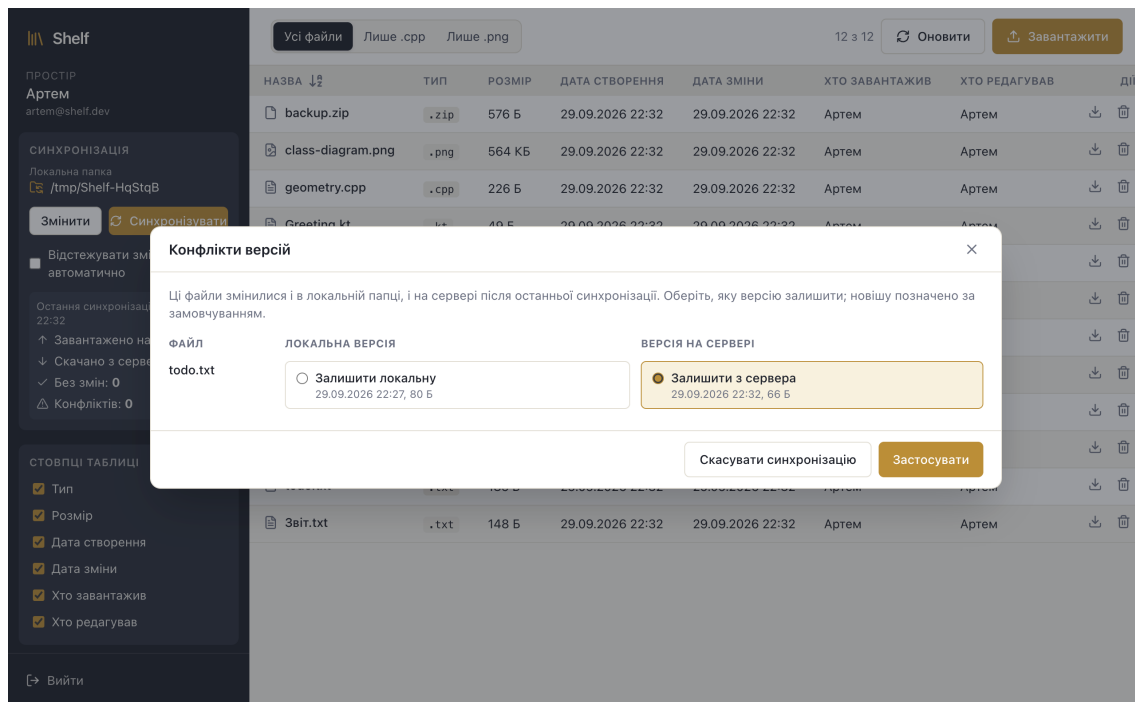


Рисунок 15. Діалог конфліктів версій

змін: 11» і «Конфліктів: 1», а `todo.txt` має розмір 66 Б. Поле «Конфліктів» рахує конфлікти, оброблені в цьому запуску, щоб було видно, що вибір застосовано. Якщо перенесення якогось файлу не вдасться, він лишиться в попередньому стані, а причина з'явиться червоним рядком під звітом.

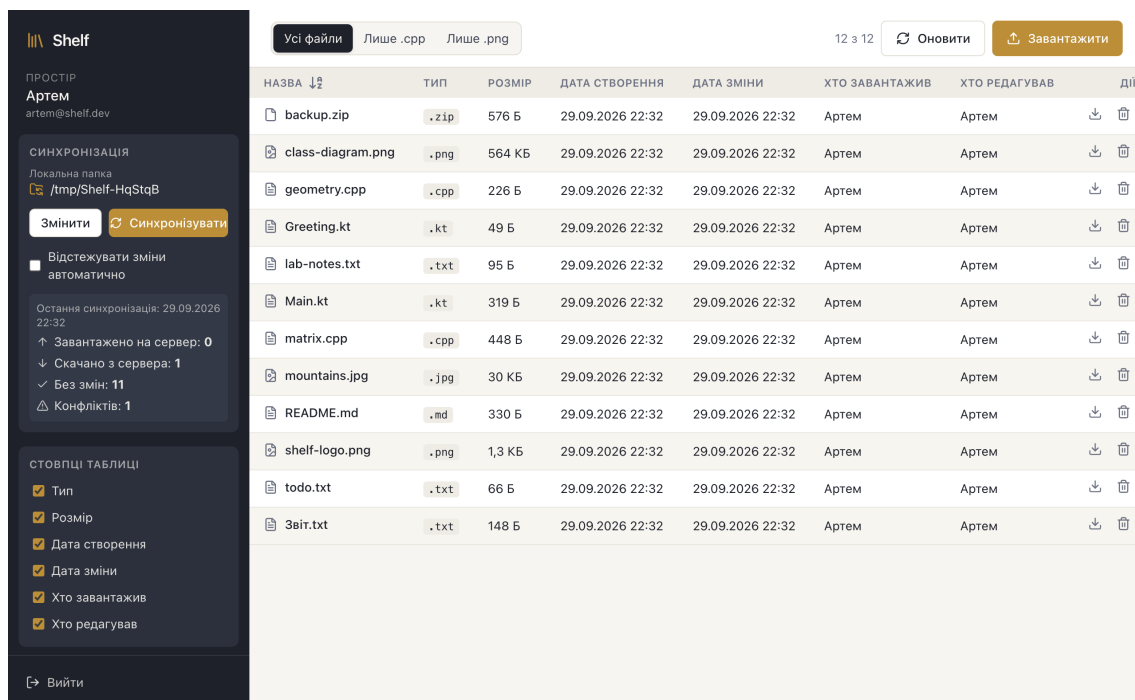
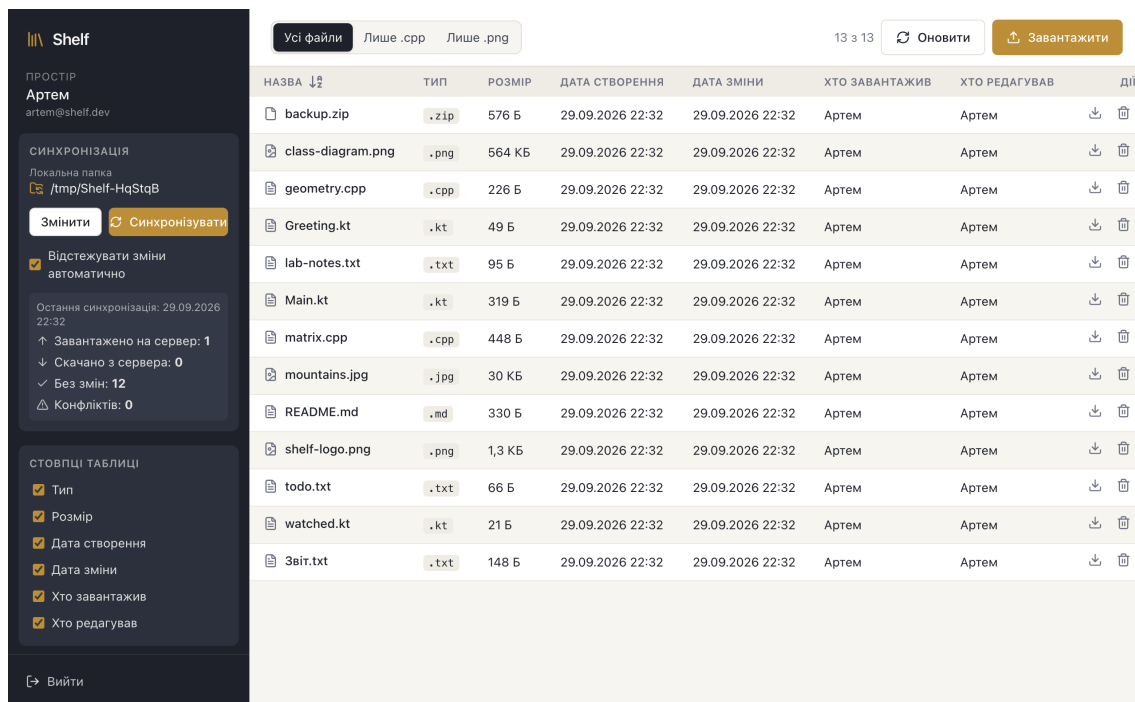


Рисунок 16. Звіт після розв'язання конфлікту

4.12 Автоматичне відстеження змін

Прапорець «Відстежувати зміни автоматично» реалізує прецедент UC14a, доступний лише в десктопі. Він запускає FolderWatcher, який через chokidar стежить за верхнім рівнем папки, пропускає приховані файли (назви з крапкою на початку, серед них і власні тимчасові .shelf-<uuid>.part) і викликає синхронізацію через 1,5 с після останньої зміни в серії. SyncService одразу виконує synchronize() без окремого сканування і без діалогу, тож конфлікт розв’язується на користь новішої версії. Якщо файли змінилися під час запуску, після нього виконується ще один; поки відкрито діалог конфліктів, автоматичні запуски чекають. На рис. 17 у папці створено watched.kt, і він сам потрапив на сервер: «Завантажено на сервер: 1», «Без змін: 12».



The screenshot shows the Shelf application interface. On the left, a sidebar displays the user 'Артем' (artem@shelf.dev) and synchronization settings. The main area shows a table of files with columns for name, type, size, creation date, modification date, uploader, and editor. The file 'watched.kt' is highlighted, showing it was uploaded and has 12 changes without conflicts.

НАЗВА	ТИП	РОЗМІР	ДАТА СТВОРЕННЯ	ДАТА ЗМІНИ	ХТО ЗАВАНТАЖИВ	ХТО РЕДАГУВАВ	ДІЇ
backup.zip	.zip	576 Б	29.09.2026 22:32	29.09.2026 22:32	Артем	Артем	↓ ↗
class-diagram.png	.png	564 КБ	29.09.2026 22:32	29.09.2026 22:32	Артем	Артем	↓ ↗
geometry.cpp	.cpp	226 Б	29.09.2026 22:32	29.09.2026 22:32	Артем	Артем	↓ ↗
Greeting.kt	.kt	49 Б	29.09.2026 22:32	29.09.2026 22:32	Артем	Артем	↓ ↗
lab-notes.txt	.txt	95 Б	29.09.2026 22:32	29.09.2026 22:32	Артем	Артем	↓ ↗
Main.kt	.kt	319 Б	29.09.2026 22:32	29.09.2026 22:32	Артем	Артем	↓ ↗
matrix.cpp	.cpp	448 Б	29.09.2026 22:32	29.09.2026 22:32	Артем	Артем	↓ ↗
mountains.jpg	.jpg	30 КБ	29.09.2026 22:32	29.09.2026 22:32	Артем	Артем	↓ ↗
README.md	.md	330 Б	29.09.2026 22:32	29.09.2026 22:32	Артем	Артем	↓ ↗
shelf-logo.png	.png	1,3 КБ	29.09.2026 22:32	29.09.2026 22:32	Артем	Артем	↓ ↗
todo.txt	.txt	66 Б	29.09.2026 22:32	29.09.2026 22:32	Артем	Артем	↓ ↗
watched.kt	.kt	21 Б	29.09.2026 22:32	29.09.2026 22:32	Артем	Артем	↓ ↗
Звіт.txt	.txt	148 Б	29.09.2026 22:32	29.09.2026 22:32	Артем	Артем	↓ ↗

Рисунок 17. Автоматична синхронізація після появи файлу в папці

5 Unit-тестування

Тести розділено на три набори за пакетами; `npm test` у корені запускає всі, а GitHub Actions — після кожного push. Кількість тестів узятو з журналів запуску; разом їх 226.

Набір	Що перевіряє	Тестів
@shelf/shared (Vitest, 11 файлів)	операції варіанта, previewKindOf і класи перегляду, стовпці, форматування, ліміт 50 МБ, FileApiClient з підміненим fetch, усі статуси computeStatus, SyncEngine з MemoryLocalFolder	140
@shelf/api (Jest, 8 наборів)	AuthService; FilesService (нова версія змінює editedBy, modifiedAt, checksum, чужий файл недоступний); StorageService з імітацією S3; конфігурація; українські помилки	58
@shelf/desktop (Vitest, 6 файлів)	SettingsStore, NodeLocalFolder, JsonSnapshotStore, FolderWatcher, SyncService; SyncEngine на справжній файловій системі з API в пам'яті	28

5.1 Обов'язкові тести операцій варіанта

Тести лежать у `fileListOperations.test.ts` і `preview.test.ts`; список `files` містить записи `b.txt`, `A.kt`, `photo.jpg`, `main.cpp`, `logo.png`, `Zeta.CPP`, `notes`.

Сортування за назвою (група «`sortByName` — операція варіанта»): за зростанням реєстр не впливає на порядок, а за спаданням результат — точне дзеркало.

```
it('sorts ascending, ignoring letter case', () => {
  expect(names(sortByName(files, SortDirection.ASCENDING))).toEqual([
    'A.kt', 'b.txt', 'logo.png', 'main.cpp', 'notes', 'photo.jpg', 'Zeta.CPP',
  ]);
});

it('sorts descending as the exact reverse', () => {
  expect(names(sortByName(files, SortDirection.DESENDING))).toEqual([
    'Zeta.CPP', 'photo.jpg', 'notes', 'main.cpp', 'logo.png', 'b.txt', 'A.kt',
  ]);
});
```

Фільтр за типом (група «`filterByType` — операція варіанта»): `ONLY_CPP` залишає `.cpp`-файли за будь-якого реєстру, `ONLY_PNG` — лише `.png`.

```
it('ONLY_CPP keeps only .cpp files, any letter case', () => {
  expect(names(filterByType(files, TypeFilter.ONLY_CPP))).toEqual(['main.cpp', 'Zeta.CPP']);
});

it('ONLY_PNG keeps only .png files', () => {
  expect(names(filterByType(files, TypeFilter.ONLY_PNG))).toEqual(['logo.png']);
});
```

Тип перегляду (група «`previewKindOf` — тип варіанта»): обов'язкові `.kt` → `TEXT` і `.jpg` → `IMAGE`, великі літери в розширенні, інші типи.

```
it.each([
  ['Main.kt', PreviewKind.TEXT],
  ['photo.jpg', PreviewKind.IMAGE],
  ['PHOTO.JPG', PreviewKind.IMAGE],
  ['main.cpp', PreviewKind.TEXT],
  ['notes.txt', PreviewKind.TEXT],
  ['logo.png', PreviewKind.IMAGE],
  ['backup.zip', PreviewKind.NONE],
  ['noext', PreviewKind.NONE],
])(('%s → %s', (name, kind) => {
  expect(previewKindOf(name)).toBe(kind);
}));
```

5.2 Результати запуску

Скорочений вивід `vitest run --reporter=verbose` для `@shelf/shared` (довгі рядки перенесено):

```
RUN v3.2.7 packages/shared
```

```
✓ src/preview.test.ts > previewKindOf — тип варіанта: .kt як текст, .jpg як зображення
  > Main.kt → TEXT 1ms
✓ src/preview.test.ts > previewKindOf — тип варіанта: .kt як текст, .jpg як зображення
  > photo.jpg → IMAGE 0ms
✓ src/fileListOperations.test.ts > sortByName — операція варіанта: сортування за назвою
  > sorts ascending, ignoring letter case 0ms
✓ src/fileListOperations.test.ts > filterByType — операція варіанта: фільтр «усі / лише
  .cpp / лише .png» > ONLY_CPP keeps only .cpp files, any letter case 0ms
```

```
Test Files 11 passed (11)
Tests      140 passed (140)
```

Підсумкові рядки Jest для сервера і Vitest для десктопа:

```
Test Suites: 8 passed, 8 total
Tests:       58 passed, 58 total
```

```
Test Files 6 passed (6)
Tests      28 passed (28)
```

Усі 226 тестів проходять; наскрізні перевірки на живому стеку виконують `pnpm smoke` (REST API) і `pnpm -F @shelf/desktop smoke` (інтерфейс, Playwright).

6 Інструкція із запуску

Потрібні Node.js 22.12 або новіший, `pnpm 10` (`corepack enable pnpm`) і Docker.

```
pnpm install
cp docker/.env.example docker/.env
pnpm stack:up           # PostgreSQL, MinIO і API
pnpm seed              # демо-користувачі та файли
pnpm -F @shelf/desktop dev  # клієнт у режимі розробки
pnpm -F @shelf/desktop dist:mac # збірка .dmg
```

Сервер слухає `http://localhost:4000` — цю адресу клієнт і пропонує на екрані входу. Готовий `.dmg` лежить в `apps/desktop/release`; він не підписаний сертифікатом Apple, тому першого разу Shelf відкривають через контекстне меню значка в теці «Програми» і пункт «Відкрити». Демо-користувачів створює `pnpm seed: artem@shelf.dev` (Артем, усі десять демонстраційних файлів), `iryna@shelf.dev` (Ірина, три файли) і `maksym@shelf.dev` (Максим, два файли); пароль у всіх `shelf-demo-2026`.

7 Висновки

На етапі 2 створено робочу десктопну версію Shelf. Сервер на NestJS зберігає метадані в PostgreSQL, а вміст файлів — у S3-сумісному сховищі; клієнт на Electron виконує вимоги R1–R9, зокрема операції варіанта 8-6 і бонусний `drag-and-drop` в обох напрямках. Синхронізація з локальною папкою знаходить конфлікти за знімком стану й дає користувачу вибрати версію. Логіку зосереджено в `@shelf/shared`, інтерфейс — у `@shelf/ui`; обидва пакети не залежать від Electron, тому веб-версія етапу 3 використає їх без змін і додасть лише свої реалізації `LocalFolder` і сховища знімка. Код перевіряють 226 unit-тестів, серед них обов’язкові тести операцій варіанта. Обмеження: видалення під час синхронізації не поширюються, а сервер зберігає лише поточну версію файлу без історії.